# HARDGRAFT — PLAN CODEX

## A. ÉTAT ACTUEL DU REPOSITORY

Repository : `Erazer88/Hardgraft`

Branche de travail actuelle : `codex/nereide-prototype`

Prototype actuel :

- HTML/CSS/JavaScript vanilla ;
- Canvas 2D ;
- aucune dépendance npm nécessaire ;
- `src/app.js` contient encore une grande partie de l'application ;
- `src/world.js` contient la génération procédurale ;
- `build.py` génère un HTML autonome ;
- tests Node ;
- smoke tests navigateur ;
- sauvegarde locale ;
- zoom continu déjà fonctionnel ;
- galaxie procédurale ;
- 2 560 systèmes ;
- système orbital ;
- terrains procéduraux ;
- brouillard et sondes simplifiées.

Le prototype actuel génère encore 4–7 planètes par système : cette règle devra évoluer.

IMPORTANT : préserver autant que possible le système actuel de navigation et de zoom continu.

---

# B. CONTRAINTE TECHNIQUE PRINCIPALE

Projet développé par :

- 1 humain non spécialiste du code ;
- Codex / agents IA.

Donc :

**LA LISIBILITÉ DU CODE EST UNE FONCTIONNALITÉ.**

Interdictions :

- gros système monolithique supplémentaire dans `app.js` ;
- logique gameplay mélangée au rendu Canvas ;
- valeurs d'équilibrage dispersées ;
- comportements implicites impossibles à tester ;
- « vraie IA » générative pour gérer les colonies ;
- framework ajouté sans justification majeure.

Préférer :

- JavaScript simple ;
- modules clairement séparés ;
- configuration data-driven ;
- fonctions déterministes ;
- tests unitaires.

---

# C. ARCHITECTURE CIBLE

Ne pas forcément tout créer immédiatement, mais converger vers quelque chose proche de :

```
src/
  app.js

  world/
    galaxy.js
    system.js
    planet.js
    terrain.js

  simulation/
    simulation.js
    resources.js
    inventory.js
    buildings.js
    factory.js
    factoryGraph.js
    production.js
    energy.js
    logistics.js

  gameplay/
```

```
    exploration.js
    research.js
    technology.js
    population.js
    context.js
    events.js
    policies.js

  data/
    resources.js
    recipes.js
    buildings.js
    technologies.js

  rendering/
    galaxyRenderer.js
    systemRenderer.js
    planetRenderer.js
    factoryRenderer.js

  ui/
    hud.js
    inspector.js
    buildMenu.js
```

Il n'est pas obligatoire d'utiliser ES modules si cela complique le HTML autonome.

Une organisation par namespaces/IIFE séparés est acceptable.

Priorité : **séparer données, simulation et rendu.**

---

# D. RÈGLES DE SIMULATION

Le rendu ne doit jamais être la source de vérité.

Séparer :

**simulation** de **animation visuelle**.

La simulation doit pouvoir être testée sans Canvas.

Les recettes, coûts et technologies doivent être déclaratifs.

Exemple conceptuel :

```
{
  id: "iron_plate",
  inputs: [{ resource: "iron_ore", amount: 2 }],
  outputs: [{ resource: "iron_standard", amount: 1 }],
  duration: 4
}
```

Les valeurs exactes sont provisoires.

Ne pas considérer les nombres temporaires comme définitifs.

---

# E. ROADMAP GÉNÉRALE

## MILESTONE 0 — Architecture saine

Pas de nouveau gameplay majeur.

Objectif : préparer le projet aux systèmes suivants.

## MILESTONE 1 — Vertical slice industriel

Une planète réellement jouable :

- ressources ;
- bâtiments ;
- construction ;
- énergie ;
- usine ;
- blocs ;
- stocks ;
- production ;
- routes/hub simplifiés.

Doit déjà être agréable à regarder et manipuler.

## MILESTONE 2 — Première boucle de progression

Ajouter :

- début avec l'Arche ;
- choix de planète ;
- coûts ;

- robots abstraits ;
- premières recherches ;
- sondes ;
- découverte progressive ;
- premier objectif interplanétaire/interstellaire.

## MILESTONE 3 — Data

Ajouter :

- datacenter ;
- réception data ;
- connexion data interne à l'usine ;
- analyse qualité ;
- matériaux avancés.

## MILESTONE 4 — Expansion

Ajouter seulement après validation des précédents :

- plusieurs planètes réellement exploitées ;
- hubs ;
- échanges ;
- premier autre système ;
- population abstraite ;
- premières traces de Contexte.

---

# F. AGENT 00 — ARCHITECTURE

## Mission

Refactoriser suffisamment le prototype pour permettre l'ajout de gameplay sans casser la navigation existante.

## Important

Cet agent travaille EN PREMIER et seul sur la structure commune.

Les autres agents attendent cette base.

## Objectifs

1. Identifier dans `app.js` :
    - état ;
    - simulation ;
    - rendu ;
    - UI ;
    - navigation.
2. Séparer progressivement ces responsabilités.
3. Conserver :
    - zoom continu ;
    - caméra ;
    - touch mobile ;
    - seed ;
    - sauvegarde ;
    - génération actuelle.
4. Créer un état de jeu extensible.
5. Ajouter les interfaces minimales pour :
    - ressources ;
    - bâtiments ;
    - usines ;
    - simulation temporelle.
6. Ne PAS implémenter encore tout le gameplay.

## Definition of Done

- prototype existant visuellement fonctionnel ;
- tests existants passent ;
- pas de régression navigation ;
- simulation future possible hors Canvas ;
- fichiers clairement séparés ;
- documentation courte de l'architecture.

---

# G. AGENT 01 — GÉNÉRATION DES SYSTÈMES

## Dépendance

Agent 00 terminé.

# Mission

Rendre les systèmes beaucoup moins uniformes.

# Ne pas modifier

le gameplay industriel.

# Objectifs

Remplacer la règle fixe 4–7 planètes par une génération capable de produire :

- système presque vide ;
- 1–2 planètes ;
- systèmes plus riches ;
- 0 ou plusieurs ceintures d'astéroïdes ;
- géantes gazeuses ;
- différents espacements orbitaux.

La distribution exacte doit être configurable.

Ne pas rendre tous les systèmes extrêmes.

Faire dépendre progressivement les caractéristiques planétaires de :

- classe stellaire ;
- distance à l'étoile.

Le système natal doit respecter des contraintes permettant une partie viable.

# Tests

Tester au minimum :

- seed déterministe ;
- diversité réelle des nombres de planètes ;
- systèmes très pauvres possibles ;
- systèmes riches possibles ;
- système natal toujours jouable.

---

# H. AGENT 02 — SIMULATION / RESSOURCES / CONSTRUCTION

## Dépendance

Agent 00 terminé.

Peut travailler parallèlement à Agent 01 si les fichiers sont séparés.

## Mission

Créer le noyau économique minimal.

## Ressources initiales

IDs stables recommandés :

- iron ;
- copper ;
- silicon ;
- carbon ;
- water.

Prévoir immédiatement la possibilité d'ajouter :

- intermédiaires ;
- déchets ;
- ressources inconnues.

## Construction

Un bâtiment possède :

- coût ;
- durée ;
- état de construction ;
- production éventuelle ;
- consommation énergétique éventuelle.

Durée initiale cible : environ 3–4 secondes.

## Démolition

Prévoir un remboursement configurable.

Valeur V0 : 100 %.

## Important

Tous les coûts doivent être dans des tables de données.

Aucun coût gameplay ne doit être enfoui dans le rendu.

## Definition of Done

Pouvoir :

- disposer d'un inventaire ;
- consommer des matériaux ;
- lancer une construction ;
- voir son timer ;
- terminer la construction ;
- démolir ;
- récupérer les ressources.

Tests unitaires sans Canvas obligatoires.

---

# I. AGENT 03 — USINE ET BLOCS

## Dépendances

Agents 00 + 02.

## Mission

Créer le premier véritable cœur de gameplay.

## Usine

Une usine :

- est placée physiquement sur la planète ;
- possède un niveau/rang ;
- possède une surface interne finie ;
- possède une consommation énergétique globale.

L'intérieur de l'usine est un canvas/espace séparé contenant des blocs flottants.

# Bloc

Chaque bloc peut posséder :

- ports entrée ;
- ports sortie ;
- stock interne ;
- capacité ;
- recette ;
- durée de cycle.

# Transferts

Quand une production termine :

si le prochain bloc a de la place : → transfert immédiat.

Sinon : → le stock reste bloqué.

# Placement

Le placement n'a aucun impact sur :

- vitesse ;
- rendement ;
- énergie.

Impact uniquement :

- lisibilité ;
- place disponible.

# Blocs V0

Implémenter seulement :

- entrée ;
- sortie ;
- transformation ;
- stockage ;
- division ;
- fusion.

Préparer l'architecture pour :

- filtre ;
- règle ;
- assemblage ;
- séparation ;
- data.

## UX

Priorité mobile :

- drag ;
- sélection ;
- création connexion ;
- suppression ;
- déplacement ;
- zoom/pan si nécessaire.

## Definition of Done

Une petite chaîne doit pouvoir réellement tourner en boucle sans UI debug.

Exemple :

Entrée minerai
→ transformation
→ stockage
→ sortie.

Les stocks doivent pouvoir créer un vrai goulot d'étranglement.

---

# J. AGENT 04 — ÉNERGIE PLANÉTAIRE

## Dépendance

Agent 02.

## Mission

Créer un système d'énergie simple.

## Règle

Pas de câbles bloc par bloc.

Une usine consomme une puissance totale.

Une planète possède :

- production disponible ;
- consommation.

Si production insuffisante : appliquer une règle simple de ralentissement ou d'insuffisance.

Les chiffres doivent être facilement ajustables.

## V0

Une seule ou deux sources énergétiques suffisent.

Ne PAS coder :

- batteries interplanétaires ;
- Dyson ;
- système énergétique avancé.

Préparer seulement les interfaces futures.

---

# K. AGENT 05 — ROUTES ET HUB PLANÉTAIRE

## Dépendances

Agents 00 + 02.

## Mission

Créer la logistique hors usine.

## Règle

Dans une usine : transfert instantané.

Sur la planète : transport avec durée.

## V0

Créer :

- hub central ;
- connexion installation → hub ;
- route terrestre simple ;
- petits véhicules animés ;
- capacité/débit simplifié.

Pas de microgestion individuelle des véhicules.

L'objectif premier est : **voir vivre le réseau.**

Les routes doivent être visuellement agréables sur le terrain.

Ne pas transformer immédiatement le terrain en puzzle de pathfinding complexe.

---

# L. AGENT 06 — RECHERCHE ET TECHNOLOGIE

## Dépendances

Simulation de base fonctionnelle.

## Mission

Créer le moteur de technologies sans arbre entièrement révélé.

## Principe

Les technologies principales sont fixes.

Mais elles apparaissent progressivement selon des triggers.

Exemples :

construction centre déchets
→ révéler recyclage
→ révéler séparation avancée

Une technologie possède :

- conditions de révélation ;
- coût de calcul ;
- durée ;
- prérequis connus ;
- effets.

## Interface

Montrer :

- technologies actuellement connues ;
- pistes accessibles ;
- silhouettes/brouillard éventuel du reste.

Ne PAS montrer tout l'arbre complet.

## Anti-RNG

Les technologies essentielles ne doivent jamais dépendre uniquement d'une découverte aléatoire.

---

# M. AGENT 07 — EXPLORATION

## Mission

Faire de la connaissance progressive une vraie mécanique.

## États d'information

Pour un monde :

1. non observé ;
2. télescope ;
3. sonde ;
4. visite physique.

Chaque état révèle davantage de propriétés.

Ne jamais révéler immédiatement toutes les ressources d'une planète éloignée.

## Événements d'exploration

Prévoir un système générique capable de signaler :

- anomalie ;
- signal ;
- épave ;
- ressource ;
- phénomène.

L'événement doit proposer de la curiosité sans devenir une quête classique.

---

# N. AGENT 08 — DATA

## Dépendances

Usines + technologies fonctionnelles.

## Mission

Créer uniquement une première vertical slice du réseau data.

## V0

Ajouter :

- datacenter ;
- émetteur/récepteur dans une usine ;
- connexion data ;
- bloc Analyse qualité.

Exemple :

Datacenter
→ Récepteur usine
→ Analyse qualité
→ bloc de fabrication.

Effet :

production normalement : **Fer standard**

avec analyse : **Fer avancé**

Fer avancé peut remplacer Fer standard dans les recettes.

Certaines recettes futures pourront exiger explicitement Fer avancé.

## Important

Ne PAS encore ajouter :

- virus ;
- paquets ;
- corruption complexe ;
- cybersécurité ;
- internet galactique complet.

Mais concevoir la data comme un véritable type de flux pour pouvoir les ajouter.

---

# O. AGENT 09 — POPULATION / VIE VISUELLE

## Dépendances

Planète et industrie fonctionnelles.

## Mission

Ajouter une présence humaine abstraite sans créer un city-builder.

## Population

Calcul simplifié à partir de :

- âge de la colonie ;
- habitabilité ;
- puissance industrielle.

Pas de :

- logement détaillé ;
- nourriture détaillée ;
- citoyens individuels.

## Visuel

Faire apparaître progressivement :

- petites zones habitées ;
- éclairage ;
- routes secondaires décoratives ;
- activité ;
- constructions civiles.

Placement naturellement privilégié :

- plaines ;
- côtes ;
- rivières ;
- zones crédibles.

La population doit surtout donner la sensation :

**la vie est en train d'apparaître.**

---

# P. AGENT 10 — ARCHE / LORE MINIMAL

## Mission

Ne PAS écrire tout le scénario.

Créer seulement le support technique.

L'Arche doit rester accessible comme :

- lieu d'origine ;
- hub de connaissance ;
- futur centre du Contexte.

Prévoir une structure pour :

- fragments d'archives ;
- découvertes ;
- corrélations ;
- textes ;
- événements narratifs.

Les textes doivent être séparés du code.

Ne pas construire de vérité définitive sur l'origine de l'Arche.

---

# Q. CE QU'IL NE FAUT PAS CODER MAINTENANT

Interdit pour la première phase :

- IA autonome concevant librement des colonies ;
- doctrines galactiques complètes ;
- rebellions complexes ;
- guerres ;
- civilisations intelligentes complètes ;
- extermination ;
- politique galactique ;
- Contexte complet ;
- batteries interstellaires ;
- Dyson ;
- trajectoires orbitales avancées ;
- économie monétaire ;
- marché ;
- milliers d'artefacts ;
- virus/data complexe ;
- simulation détaillée de centaines de systèmes actifs.

Ces mécaniques font partie de la direction future.

Elles ne doivent pas parasiter la première boucle jouable.

---

# R. PREMIER OBJECTIF JOUABLE

Avant toute extension complexe, Hardgraft doit permettre ceci :

1. ouvrir une partie ;
2. voir l'Arche ;
3. observer plusieurs mondes ;
4. sélectionner une planète ;
5. commencer avec quelques ressources/robots ;
6. construire une énergie ;
7. extraire fer/cuivre/etc. ;
8. poser une usine ;

9. ouvrir l'usine ;
10. créer une chaîne de blocs ;
11. voir réellement les stocks circuler ;
12. produire un composant utile ;
13. construire de nouvelles installations ;
14. voir des routes et véhicules ;
15. améliorer progressivement la planète ;
16. recevoir une première piste technologique ;
17. lancer une sonde ;
18. découvrir quelque chose donnant envie de continuer.

Si ces 18 étapes ne sont pas satisfaisantes :

**NE PAS PASSER À L'AUTOMATISATION GALACTIQUE.**

---

# S. TESTS OBLIGATOIRES

Chaque agent doit ajouter des tests adaptés.

Au minimum :

### Monde

- reproductibilité seed ;
- variété.

### Simulation

- conservation des ressources ;
- consommation construction ;
- remboursement ;
- production.

### Usines

- capacité stocks ;
- blocage ;
- transfert ;
- division/fusion ;
- cycles.

### Technologie

- révélation correcte ;
- recherche ;

- application globale.

### Data

- standard sans analyse ;
- avancé avec analyse ;
- avancé accepté dans recette standard.

Les simulations doivent être testables sans navigateur.

---

# T. RÈGLES POUR LES AGENTS

Avant de coder :

1. lire ce document ;
2. lire le README ;
3. inspecter les fichiers concernés ;
4. identifier les tests existants ;
5. annoncer les fichiers qui seront modifiés.

Pendant :

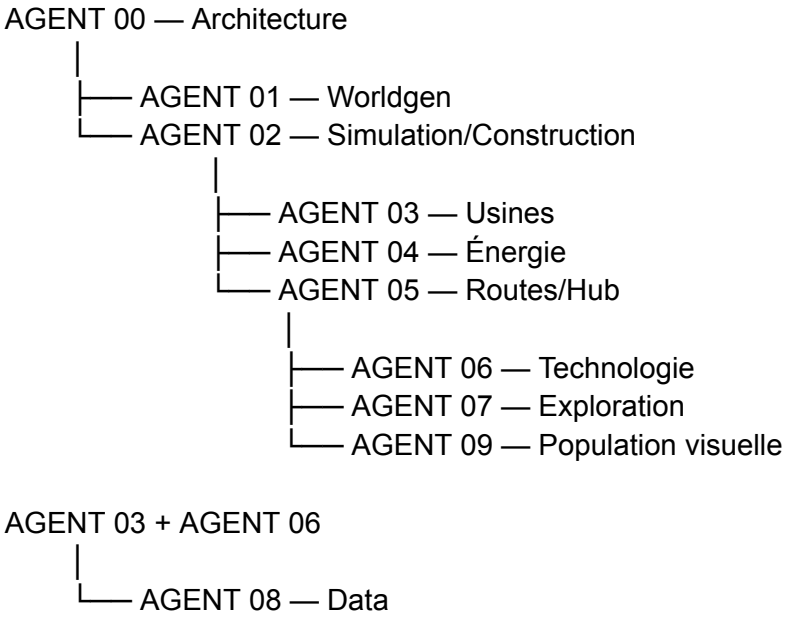
- ne pas réécrire des systèmes fonctionnels sans nécessité ;
- modifier le minimum de fichiers communs ;
- éviter les dépendances externes ;
- conserver le mobile ;
- conserver le build standalone.

Après :

1. exécuter les tests unitaires ;
2. exécuter les smoke tests pertinents ;
3. vérifier desktop ;
4. vérifier viewport mobile ;
5. résumer précisément :
    - fichiers modifiés ;
    - comportement ajouté ;
    - limites ;
    - prochaines dépendances.

---

# U. ORDRE RECOMMANDÉ DES AGENTS




Plus tard :
AGENT 10 — Arche/Lore

Ne pas lancer deux agents parallèles sur les mêmes fichiers.

Après chaque groupe : faire une intégration et un test avant de poursuivre.

---

# V. PREMIÈRE COMMANDE À DONNER À CODEX

Commence uniquement par **Agent 00**.

Prompt :

« Lis intégralement le Game Design Source of Truth Hardgraft et le repository actuel. Ta mission n'est pas d'ajouter de gameplay : prépare l'architecture pour la simulation industrielle à venir tout en préservant strictement la navigation, le zoom continu, la génération procédurale, le fonctionnement mobile et le build HTML autonome.

Inspecte d'abord le code et propose un plan de refactorisation minimal. Sépare progressivement monde, simulation, rendu et données sans introduire de framework ni de dépendance inutile. Conserve tous les tests existants et ajoute les tests nécessaires.

Ne code aucune IA autonome, économie avancée, lore complet, industrie complète ou mécanique future. La priorité absolue est d'obtenir une base lisible qu'un développeur non spécialiste pourra continuer à faire évoluer avec Codex.

À la fin, fournis :

1. architecture obtenue ;
2. liste exacte des fichiers modifiés ;
3. tests exécutés ;
4. régressions éventuelles ;
5. interfaces préparées pour ressources, bâtiments et usines. »